Blood Bank Management System: A Role-Based Full-Stack Platform for Real-Time Inventory, ABO/Rh Compatibility Matching, and Emergency Donor Alerting

Mrs. Anitha M
Dept. of CSE,
Rajalakshmi Engineering College
Chennai, India
anitha.muthulingam@rajalakshmi.edu.in

Bharath Kumar M (230701053)
Dept. of CSE,
Rajalakshmi Engineering College
Chennai, India
230701053@rajalakshmi.edu.in

Akshay Kumar S (23071021)
Dept. of CSE,
Rajalakshmi Engineering College
Chennai, India
230701021@rajalakshmi.edu.in

Bernieo Fatim A (230701051)
Dept. of CSE,
Rajalakshmi Engineering College
Chennai, India
230701051@rajalakshmi.edu.in

Ashish P Shaji (230701041)
Dept. of CSE,
Rajalakshmi Engineering College
Chennai, India
230701041@rajalakshmi.edu.in

***Abstract*—Blood is perishable and life-critical, yet most blood banks still run on paper registers, spreadsheets, single-branch software, or ad-hoc phone tools, with no cross-facility visibility, ABO/Rh verification before issuance, or audit trail. This paper presents the Blood Bank Management System (BBMS), a role-based web platform built with Spring Boot 3 (Java 17), React 18, and MySQL 8 that digitizes the donation lifecycle from eligibility screening and blood-drive intake through hospital verification, ABO/Rh-aware request fulfillment, transfusion issuance, unit-level shelf-life tracking, and audit logging. Beyond the architecture, we evaluate three internal mechanisms experimentally: the donor eligibility engine (622 boundary cases, 100% agreement with independently derived expectations); the ABO/Rh matcher (all 64 AABB recipient–donor pairs, 100% agreement, plus 200 randomized ranking trials); and optimistic-locking inventory control (at 200 concurrent transactions a naive read-modify-write lost a mean of 132 updates, versus 0.2 with optimistic locking, at a cost of about 1,558 retries). End-to-end latency of the Twilio alerting pipeline was not measured and is identified as future work.**

***Keywords*—Blood bank management, healthcare information systems, ABO/Rh compatibility, donor eligibility, role-based access control, JWT, Twilio, Spring Boot, React, WebSocket, audit trail.**

I. INTRODUCTION

Coordination among donors, hospitals, and blood banks is one of the most delicate links in emergency healthcare. Blood banks still commonly rely on paper registers, spreadsheet logs, single-branch desktop or PHP applications, or phone/WhatsApp coordination during shortages [1], [2]. None of these gives a hospital a real-time view of cross-facility inventory or enforces compatibility constraints before a unit is issued. Because blood is perishable and transfusion errors are dangerous, any delay in finding a matching, eligible donor directly raises patient risk and wastes units.

BBMS digitizes the end-to-end donation workflow on one rule-enforced platform, targeting UN Sustainable Development Goal 3 (Good Health and Well-Being). It is a role-based full-stack application (Spring Boot REST API, React frontend, MySQL) in which inventory changes, hospital requests, and transfusion issuance can occur only through server-enforced operations, never arbitrary edits. A WebSocket (STOMP) channel pushes emergency alerts, inventory changes, and expiry warnings to subscribed dashboards without polling.

The contributions are: (i) a deterministic server-side donor eligibility engine; (ii) an eight-group ABO/Rh matcher that ranks compatible stock and donors; (iii) a working emergency gateway that places Twilio voice calls and SMS to on-call donors rather than merely recording a notification; (iv) barcoded, component-aware unit-level expiry tracking without RFID hardware; and (v) experimental evidence for the eligibility, matching, and concurrency mechanisms.

II. RELATED WORK

Manika et al. [1] built a PHP/MySQL donor registry and inventory system that cut search and registration time but did not automate compatibility or eligibility screening. Shahakar et al. [2] offered a donor–hospital portal centered on directory lookup, and Iyer et al. [3] proposed a cloud-hosted multi-branch architecture without real-time matching or alerting. Ben Elmir et al. [4] forecast demand with machine learning, while blockchain approaches [5]–[7], [15] add tamper-proof records and donor privacy at the cost of running a distributed ledger. Ismael et al. [8] found systematic gaps in eligibility automation and interoperability across smart blood-bank systems.

Predictive work includes donor-willingness models [9], eligibility classifiers [10], [11], and deep reinforcement learning for platelet inventory [12]. Pavan Kumar et al. [13] combined RFID with AI-based priority queuing, reporting roughly 40% faster emergency response, and Kim et al. [14] used RFID sensor networks for cold-chain monitoring. Table I compares the reviewed systems on four axes: real-time cross-facility inventory, ABO/Rh matching, rule-based eligibility, and live emergency alerting.

Within this limited review, none covered all four. BBMS deliberately stays with a conventional relational, stateless-REST design, avoiding ledger and RFID overhead, and closes the last-mile gap of actually contacting ranked donors through an ordinary telephony API.

TABLE I
COMPARISON OF REVIEWED SYSTEMS

System	Real-time inv.	ABO/Rh match	Eligibility	Emergency alert
Manika [1]	Partial	No	No	No
Shahakar [2]	No	No	No	No
Iyer [3] (cloud)	Yes	No	No	No
BloodChain [5]	Partial	No	No	No
Pavan Kumar [13]	Yes (RFID)	No	No	Partial
BBMS (this work)	**Yes**	**Yes (8 groups)**	**Yes**	**Voice + SMS**

III. PROBLEM STATEMENT AND SCOPE

In the existing manual setting, staff cannot see whether a nearby bank holds a required group; eligibility (age, weight, hemoglobin, cooldown) is checked by hand; no ABO/Rh verification precedes issuance; hospitals are not authorized before raising requests; there is no audit trail; inventory is kept as aggregate counts so near-expiry units go unnoticed; and emergency donor calls are made manually. No integrated platform tracks inventory in real time, screens compatibility and eligibility, tracks expiry per unit, and raises emergency alerts under one auditable system.

Phase-I scope covers authentication for donors, hospitals, and blood banks; the eligibility engine; inventory and ABO/Rh matching; blood-drive intake; unit-level expiry tracking; transfusion issuance; audit and reporting; a WebSocket layer; and Twilio voice/SMS alerts. National registry integration (e.g., NACO) and a native mobile app are deferred. Four non-functional properties shaped the design: *safety* (pure, unit-testable matching and eligibility logic that audits every state change), *availability of critical alerts* (if Twilio is unreachable the alert is still recorded in-app and over WebSocket), *statelessness* (JWT authentication permits horizontal scaling), and *least-privilege data exposure* (PII is filtered by role inside the service layer).

IV. SYSTEM ARCHITECTURE

BBMS is a three-tier system (Fig. 1). The presentation layer is a React 18 SPA (Vite, Tailwind CSS, React Router v6) that calls the backend over HTTPS/REST with Axios. The application layer is a Spring Boot 3.3.3 REST API on Java 17 (Spring Web, Security, Data JPA, WebSocket, Validation; JJWT 0.12.6; SpringDoc OpenAPI) organized into twelve business modules. The data layer uses Hibernate/JPA over MySQL 8, with an in-memory H2 database seeded automatically for development and evaluation. Twilio's Voice and SMS REST APIs are called directly over HTTP rather than through its SDK. Testing uses JUnit 5 and Spring Security Test.

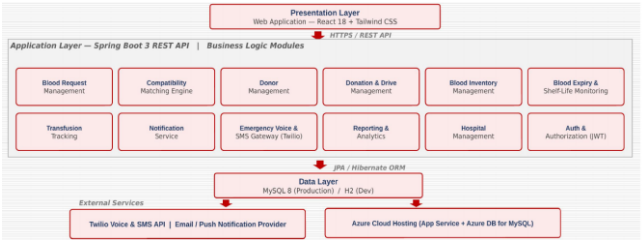


Fig. 1. Layered architecture of BBMS: React presentation layer, twelve Spring Boot business modules, MySQL/H2 data layer, and external Twilio and Azure services.

TABLE II
TECHNOLOGY STACK OF BBMS

Layer	Technology
Backend runtime	Java 17, Spring Boot 3.3.3 (Web, Security, Data JPA, WebSocket, Validation)
Auth	JJWT 0.12.6 (JSON Web Tokens), BCrypt password hashing
API docs	SpringDoc OpenAPI 2.6.0 (Swagger UI)
Persistence	Hibernate ORM; MySQL Connector/J (prod); H2 in-memory (dev)
Frontend	React 18, Vite, Tailwind CSS, Lucide icons, React Router v6
Networking	Axios (REST); SockJS + STOMP WebSocket client
Emergency alerts	Twilio Voice and SMS REST API via RestTemplate (no SDK)
Testing / build	JUnit 5, Spring Security Test; Maven 3.9.6, npm
Hosting	Azure App Service, Static Web Apps, Azure Database for MySQL

The target deployment is on Microsoft Azure: traffic enters through Azure Front Door/CDN, which fans out to an Azure Static Web App hosting the React build and an Azure App Service (Linux, Java 17) hosting the Spring Boot JAR, which connects to Azure Database for MySQL (Flexible Server, SSL). A GitHub Actions workflow builds and redeploys both on every push to the main branch. Secrets such as the datasource credentials and JWT key are supplied as App Service environment variables rather than committed to code.

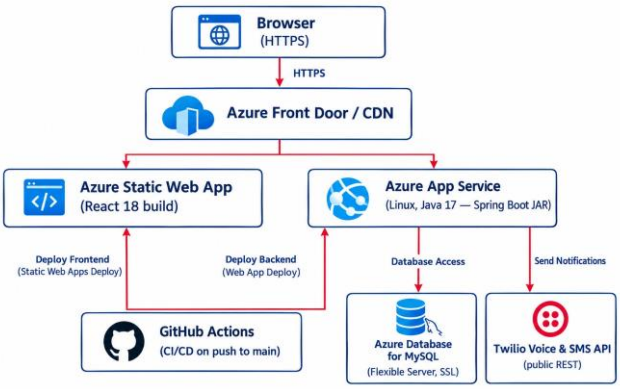


Fig. 2. Target Azure deployment: Front Door/CDN routes to Static Web Apps and App Service, which persists to Azure Database for MySQL and calls Twilio; GitHub Actions drives CI/CD.

The runbook provisions a resource group, an Azure Database for MySQL Flexible Server (Burstable tier, MySQL 8.0.21, 32 GB), a Linux B1 App Service plan on the Java 17 runtime, and an Azure Static Web App built from the frontend directory. Locally, the same codebase runs against H2 (spring.profiles.active=dev), auto-seeded by DataInitializer with five demonstration accounts (one per role plus a second donor) and two blood banks, so the whole system can be evaluated with two commands and no external database.

TABLE III
SEEDED DEMONSTRATION ACCOUNTS

Role	Login	Demonstrates
System Admin	admin@bbms.org	Global metrics, hospital approval, audit stream
Blood Bank Admin	metro.admin@bbms.org	Live stock matrix, transfusion issuance
Hospital	city.general@hospital.org	Verified account, raising requests
Donor (O−)	sarah.donor@example.com	Universal donor, digital pass
Donor (O+)	john.donor@example.com	Standard eligible donor

V. DATA MODEL

Persistence is modeled as twelve JPA entities (Table IV), each mapped to an indexed table with explicit foreign keys

and unique constraints. The inventory table carries a Hibernate @Version column for optimistic locking. Enumerations constrain state at compile time (eligibility, request urgency, unit status, roles). The BloodGroup enum carries the ABO/Rh compatibility matrix as behavior, and the BloodComponent Type enum carries each product's clinically correct shelf life and storage requirement (Table V).

TABLE IV
CORE JPA ENTITIES AND KEY CONSTRAINTS

Entity	Purpose / key constraints
User	Account (email, BCrypt hash, role); one-to-one with Donor or Hospital
Donor	Blood group, DOB, weight, Hb, last donation, health flags, eligibility status; indexed
Hospital	License number, verification status; must be VERIFIED to raise requests
BloodBank	Facility; owns inventory, units, drives
BloodInventory	Units per (bank, group); unique pair; @Version locking
BloodUnit	Barcoded unit (unique number); component, collection/ expiry dates, status
BloodRequest	Group, units needed/fulfilled, urgency, status
Transfusion	Issuance linked to a request; actor id/role/name captured
DonationDrive / DriveRegistration	Scheduled collection event; donor sign-ups per drive
Notification	In-app/SMS/voice record with delivery status
AuditLog	Entity, action, details, actor and timestamp; indexed

TABLE V
BLOOD COMPONENT SHELF LIFE AND STORAGE

Component	Shelf life	Volume	Storage
Packed red cells	42 days	450 mL	1–6 °C refrigerator
Whole blood	35 days	450 mL	1–6 °C refrigerator
Platelet concentrate	5 days	250 mL	20–24 °C agitator
Fresh frozen plasma	365 days	250 mL	−18 °C or colder
Cryoprecipitate	365 days	100 mL	−18 °C or colder

A User specializes into a Donor, a Hospital, or bank staff. A BloodBank has many BloodInventory rows (one per group), BloodUnits, and DonationDrives; a Hospital has many BloodRequests, each of which may be met by several Transfusions as stock arrives. AuditLog rows shadow every create or mutate. Repositories mix derived queries with a few explicit JPQL queries: a join-fetch avoids N+1 loading when listing donors, and aggregate and expiry-window queries (24 h, 48 h, 7 days; grouped counts) run in the database rather than in Java.

VI. CORE MODULE DESIGN

A. Authentication and Role-Based Access Control

Login at POST /api/v1/auth/login returns an HMAC-signed JWT (24-hour expiry) carrying user id, email, and role. A custom JwtAuthenticationFilter validates it ahead of Spring Security's own filter and builds a UserPrincipal for @PreAuthorize checks; passwords are stored only as BCrypt hashes. Only genuinely public endpoints (auth, Swagger, the compatibility matrix, public drive and bank listings, a public summary, Twilio webhooks, and the WebSocket handshake) skip authentication. Method security restricts sensitive operations, such as hospital verification, to SYSTEM_ADMIN.

B. Donor Eligibility Engine

DonorService.evaluateEligibility() is a deterministic function from donor attributes to an EligibilityStatus, a boolean flag, and a list of reasons. A donor must be 18–65 years old (inclusive), weigh at least 50 kg, have hemoglobin of at least 12.5 g/dL if recorded, and be more than 90 days past the last donation (the engine also returns the next eligible date), with none of four self-reported flags set (recent major surgery, pregnancy, active infection, tattoo/piercing within six months). Failing only on a surgery or infection flag yields PENDING_REVIEW, since those are temporary; an administrator may override with a recorded reason that enters the audit trail. Every profile update recomputes eligibility; Table VI lists sample unit tests, each exercising one disqualification path. Contact data is visible only to administrators and the donor; others, including hospitals searching for matches, see an anonymized "Donor #N".

```
evaluateEligibility(donor):
 age<18 or age>65 -> reason "age 18-65"
 weight<50 -> "min 50 kg"; Hb<12.5 -> "min 12.5"
 daysSinceLastDonation<90 -> "cooldown"
 surgery or infection -> PENDING REVIEW
 pregnancy or tattoo   -> reason added
 return reasons.isEmpty()? ELIGIBLE : NOT_ELIGIBLE
```

TABLE VI
SAMPLE UNIT TESTS FOR THE ELIGIBILITY ENGINE

Test case	Input	Expected result
Fully eligible	Age 25, 70 kg, Hb 14.0, last donation 100 d ago	ELIGIBLE, no reasons
Under-age	Age 17	NOT_ELIGIBLE, age reason
Under-weight	47 kg	NOT_ELIGIBLE, 50 kg reason
Cooldown	Last donation < 90 d ago	NOT_ELIGIBLE, next date returned
Recent surgery	recentMajorSurgery = true	PENDING_REVIEW

C. ABO/Rh Compatibility Matching

Compatibility is computed from BloodGroup.getCompatibleDonors() and getCompatibleRecipients(), which encode the standard red-cell matrix for all eight groups (O− universal donor, AB+ universal recipient); Table VII reproduces it. For a target group and quantity, CompatibilityMatchingService. findMatches() resolves the compatible donor groups, queries live inventory across banks, and ranks stock by exact-group match first and highest availability second. It then ranks eligible donors by exact match first and longest time since last donation second, so the longest-rested donors are contacted first. The response reports total compatible stock, whether the request is fully satisfiable, and both ranked lists, with donor PII shown only to administrators.

TABLE VII
ABO/RH RECIPIENT–DONOR COMPATIBILITY

Recipient	Compatible donor groups
O− / O+	O− | O−, O+
A− / A+	O−, A− | O−, O+, A−, A+
B− / B+	O−, B− | O−, O+, B−, B+
AB− / AB+	O−, A−, B−, AB− | all eight groups

D. Inventory, Unit Lifecycle, Drives, and Requests

BloodInventory holds an aggregate count per (bank, group), guarded by a unique constraint and the @Version field. Under a zero-direct-mutation invariant, no endpoint sets stock directly; only drive batch-confirmation (adds stock) and transfusion issuance (deducts stock) change it, each writing an AuditLog row. Each physical bag is also a BloodUnit row with a barcode-like number (e.g., BAG-NYC-2024-00101), component, dates, and status (AVAILABLE, EXPIRING_ SOON, EXPIRED, TRANSFUSED, DISCARDED), enabling per-bag expiry alerts. A donation drive records title, location, time window, and target; donors pre-register and a bank operator confirms a batch, marking each registration COMPLETED or REJECTED, adding stock, and resetting that donor's 90-day cooldown from the moment of collection. In this release BloodUnit records are a seeded set rather than created at drive confirmation, which is noted as future work. A verified hospital raises a BloodRequest with group, units, and urgency (ROUTINE, URGENT, CRITICAL); the service rejects unverified hospitals regardless of frontend validation,

and unitsFulfilled tracks partial fulfillment across several transfusions.

E. Transfusion, Audit, and Expiry Monitoring

Each Transfusion row records the request, issuing bank, group, units, patient, and the id, role, and name of the acting user. All state-changing modules write through a shared AuditService (GET /api/v1/audit), which labels scheduled jobs as SYSTEM and swallows persistence exceptions so a logging outage cannot become an availability outage, making the audit trail best-effort rather than transactionally guaranteed. BloodExpiryMonitorService reports units expiring within 24 h, 48 h, and 7 days, expired units pending discard, platelet-specific risk, breakdowns by component and group, and a wastage-prevention rate. A daily job (cron 0 0 8) marks units past expiry EXPIRED and those within three days EXPIRING_SOON, and a proactive scan broadcasts a summary (e.g., "3× O− Platelets") to hospitals and on /topic/expiry-alerts so near-expiry stock can be used first.

F. Emergency Voice/SMS Gateway and Real-Time Layer

TwilioVoiceService places outbound calls by calling the Twilio REST API over HTTP with Basic authentication. Because trial accounts reject inline TwiML, the service passes a URL-encoded script (recipient, blood group, requesting hospital, repeated once) to the public twimlets.com echo utility. It normalizes Indian and US phone formats to E.164 and degrades to a clearly marked simulated response when Twilio is disabled, so the workflow stays testable. A VoiceTwimlController can serve dynamic TwiML for full Twilio accounts. EmergencyCallRequestDto requires only a phoneNumber, with optional recipientName, bloodGroup, hospitalName, customMessage, voiceUrl, donorId, and requestId, so one endpoint serves both a matching-linked call and an ad-hoc manual call. The response DTO mirrors Twilio's call SID, status, and dialed numbers so failures (invalid number, exhausted credit, carrier rejection) appear as specific statuses. For real-time delivery, an in-memory STOMP broker exposes /ws (with SockJS fallback), and RealtimeEventPublisher stamps events with a UUID, timestamp, and urgency before publishing to /topic/emergency-alerts, /inventory-updates, /blood-requests, /expiry-alerts, and per-user queues, so every dashboard updates live.

G. Critical-Path Sequence

Fig. 3 traces a hospital raising a CRITICAL request. The controller stores the request and calls the matching engine synchronously. If stock is insufficient, the notification module sends dual-channel SMS and voice alerts to top-ranked donors and broadcasts on /topic/emergency-alerts so all dashboards see the emergency, while an AuditLog row is written at creation independently of matching time.

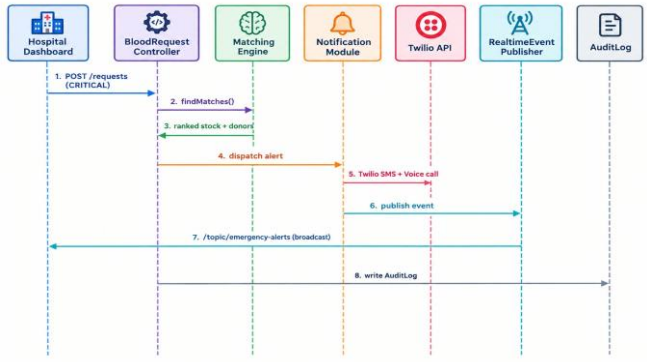


Fig. 3. Critical-path sequence from a CRITICAL hospital request to matching, dual-channel alert, live broadcast, and audit logging.

VII. REST API AND FRONTEND

The backend exposes 56 REST endpoints across 14 @Rest Controller classes under /api/v1, documented via SpringDoc/ OpenAPI and a Postman collection. DriveController has the most (9), followed by DonorController (6) and HospitalController (5). List endpoints are scoped by caller identity (/donors/me, /requests/my) or restricted to administrative roles, and a few endpoints are deliberately public so a landing page can show live content without exposing PII. A GlobalExceptionHandler maps all exceptions to one ApiResponse envelope so the frontend can branch on HTTP status alone (Table IX). Table VIII lists endpoints per controller.

TABLE VIII
REST ENDPOINTS PER CONTROLLER

Controller	Count	Representative endpoints
Drive	9	POST /drives, GET /drives/upcoming, POST /drives/{id}/confirm-batch
Donor	6	GET /donors/me, POST /donors/{id}/override-eligibility
Hospital	5	GET /hospitals/pending, PUT /hospitals/{id}/verify
BloodRequest	5	POST /requests, GET /requests/my
Notification	5	POST /notifications/emergency-call
BloodBank / BloodExpiry / Reporting	4 each	/blood-banks, /expiry/analytics, /reports/dashboard
Auth / Inventory / Transfusion	3 each	/auth/login, /inventory/summary, POST /transfusions
Matching / Audit	2 each	POST /matching/find, GET /audit
VoiceTwiml	1	GET /voice/twiml

TABLE IX
EXCEPTION-TO-HTTP STATUS MAPPING

Exception	Status	Typical trigger
ResourceNotFoundException	404	Unknown donor, hospital, or request id
BadRequestException / validation	400	Invalid payload or constraint failure
InsufficientInventoryException	409	Issuance exceeds available stock
Unauthorized / BadCredentials	401	Missing, expired, or invalid JWT
AccessDeniedException	403	@PreAuthorize role check failed

As a concrete contract, a POST /api/v1/matching/find call for two units of O− returns the total compatible stock, a fully Satisfiable flag, and the ranked stock and donor lists (donor identifiers illustrative):

```
POST /api/v1/matching/find
{ "bloodGroup": "O_NEGATIVE", "unitsNeeded": 2 }
200 OK { "totalCompatibleStock": 18,
  "fullySatisfiable": true,
  "rankedStock": [{"bloodBank": "Metro Central",
     "unitsAvailable": 18}],
  "rankedDonors": [{"donorId": 12,
     "daysSinceLastDonation": 141}] }
```

The frontend is one Vite/React 18 application with four role-specific dashboards behind shared landing, login, and register routes. AuthContext holds the JWT and role and feeds a ProtectedRoute wrapper; WebSocketContext opens one STOMP connection and exposes typed subscribe helpers. Server state lives in local component state populated by Axios and WebSocket events. Two contexts were used because authentication changes once per session while WebSocket events can arrive dozens of times per minute during an emergency; components subscribe only to their own topics to avoid needless re-renders. Tailwind's responsive classes collapse tile grids on narrow screens, modals trap focus and close on Escape, and state badges pair color with text labels for color-blind users.

VIII. SECURITY ARCHITECTURE

Security has four layers. At the transport/session layer the API is stateless, CSRF protection is disabled in favor of token authentication, and CORS is limited to an explicit origin allow-list. At the authentication layer, the JWT filter verifies signature and expiry on every request. At the authorization layer, route rules and @EnableMethodSecurity require specific roles for operations such as hospital verification and eligibility override. At the data layer, donor and hospital contact data is filtered inside the service layer, so a client that ignores frontend masking still cannot read it. Request DTOs use Jakarta Bean Validation (e.g., @NotBlank, @Email, and @Min(1) on unitsNeeded), and because all access goes through parameterized JPA queries, first-order SQL injection is closed by construction.

A STRIDE-style review found spoofing mitigated by signed JWTs, tampering with stock by the zero-direct-mutation design, repudiation by the actor-attributed audit trail, information disclosure by role-based PII filtering, and privilege escalation by @PreAuthorize checks. Denial of service on the alerting path is only partly mitigated by graceful degradation, so rate-limiting in front of notification endpoints is still needed. One configuration issue was found: the public route list covers all of /api/v1/notifications/**, intended for Twilio webhooks. The other endpoints under that prefix remain protected by method-level checks and getCurrentUser(), so no extra disclosure results, but the allow-list should be narrowed to the webhook paths before production.

IX. USER INTERFACE WALKTHROUGH

At registration users choose Donor, Hospital, or Blood Bank Staff; donors also enter blood group, date of birth, weight, hemoglobin, and the four health flags so an eligibility verdict exists immediately. The donor portal (Fig. 6) shows the live verdict, the values behind it, a Digital Donor Pass, and open drives. The Blood Bank Admin dashboard (Fig. 4) centers on a Real-Time Stock Matrix with per-group counts and ADEQUATE/LOW STOCK badges, plus pending requests, batch intake, and an Emergency Mobile Call modal that exposes the exact voice script sent through Twilio. The hospital dashboard (Fig. 5) offers a Raise Blood Request form and a live request list with fulfillment progress bars. The System Admin's Global Admin Center (Fig. 7) shows platform-wide stock, donor, hospital, and transfusion metrics with verification queues and the live audit stream, and the Shelf-Life view (Fig. 8) lists tracked units with expiry countdowns, a platelet risk count, and the wastage-prevention rate.

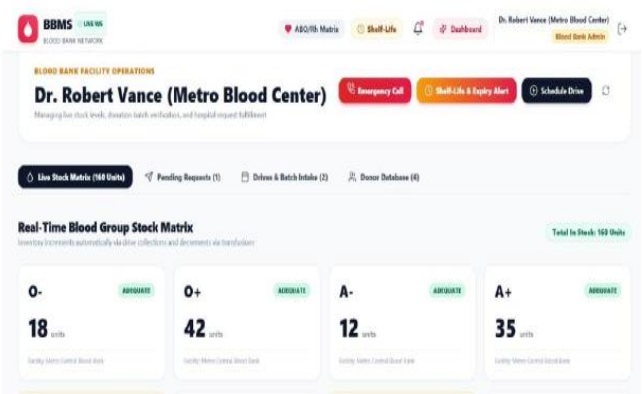


Fig. 4. Blood Bank Admin dashboard: Live Stock Matrix with per-group counts and stock badges.

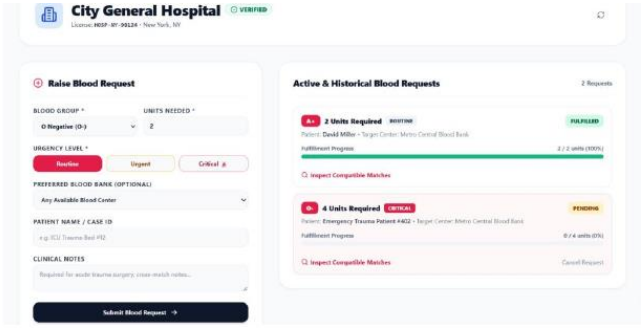


Fig. 5. Hospital dashboard: raising a blood request and tracking fulfillment of active and past requests.

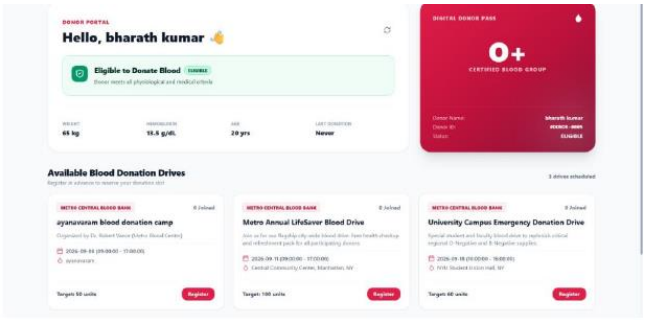


Fig. 6. Donor dashboard: live eligibility status, Digital Donor Pass, and open donation drives.

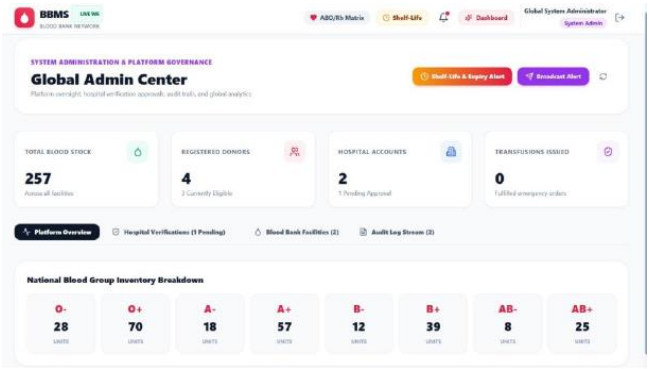


Fig. 7. System Admin Global Admin Center: platform-wide stock, donor, hospital, and transfusion metrics with the national blood-group breakdown.

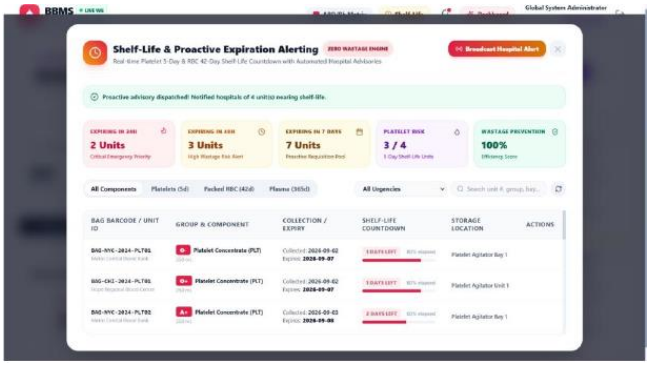


Fig. 8. Shelf-Life and Proactive Expiration Alerting dashboard listing individually tracked units with expiry countdowns.

A full critical-request episode, run on the seeded demo accounts, behaved as designed: a hospital submits a 4-unit O− CRITICAL request; the matching engine finds insufficient local stock and ranks donors; SMS and voice calls go to the top candidates and a live broadcast updates every open dashboard; a donor attends, the bank confirms the donation (stock increases, cooldown resets); the bank issues the transfusion (stock decrements, Transfusion and AuditLog rows are written); and the request card shows FULFILLED at 100%. This is a functional verification on a local deployment, not a measured trial.

X. EXPERIMENTAL METHODOLOGY AND RESULTS

Three mechanisms were evaluated using line-referenced ports of the deployed decision procedures (evaluateEligibility(), the BloodGroup compatibility methods, and the read-then-conditional-write pattern of InventoryService) in an isolated harness, because the sandboxed environment could not reach Maven Central to build the live Spring Boot and MySQL stack. The project's JUnit 5 suite (DonorEligibility Test, MatchingEngineTest, TwilioVoiceServiceTest) and Postman collection, which cover all 56 endpoints, run against the real compiled service and complement these experiments.

A. Experiment 1: Eligibility Engine

A 622-case suite covers every threshold: 100 varied eligible profiles; exact-boundary cases (ages 18 and 65, weight 50.0 kg, Hb 12.5 g/dL, 90-day cooldown); 50 cases per single-condition violation; 40 multiple-violation cases; 20 mixed-severity cases; and cases with missing fields. Expected results were fixed from the rule specification independently of the code, so the check is genuine rather than self-consistent. All 622 cases (100%) matched. One earlier failure traced to a flawed test oracle (a subject exactly 65 years and one day old, correctly admitted by age ≤ 65); the case was corrected and the rule left unchanged.

B. Experiment 2: ABO/Rh Matching

getCompatibleDonors() was checked against the AABB Technical Manual (20th ed.) reference matrix covering all 8×8 = 64 pairs, and findMatches() ranking was tested with fixed and randomized cases (Table X). Each of the 200 randomized trials drew a random target group and a shuffled, randomly sized inventory over its compatible groups; the exact-match-first invariant held in all of them.

TABLE X
ELIGIBILITY AND MATCHING VALIDATION RESULTS

Test	Cases	Correct
Eligibility: eligible profiles	100	100
Eligibility: age / weight / Hb / cooldown boundaries	260	260
Eligibility: health flags and combined violations	260	260
Eligibility: missing fields	2	2
Eligibility total	**622**	**622 (100%)**
Matching: 64-pair AABB matrix	64	64 (100%)
Matching: fixed ranking and sufficiency cases	5	5
Matching: randomized exact-match-first	200	200 (100%)

C. Experiment 3: Inventory Concurrency

To test whether @Version prevents lost updates, N concurrent OS processes (Python multiprocessing, not coroutines) each deduct 1 from a shared counter initialized to N; the correct final value is 0, so any positive value is a lost update. A naive mode reads, sleeps to simulate request latency, and blind-writes stock−1. An optimistic mode reads stock and version, sleeps, and commits only by compare-and-swap if the version is unchanged, retrying on conflict, mirroring UPDATE … WHERE version = ?. Table XI and Fig. 9 give the results.

TABLE XI
LOST UPDATES UNDER NAIVE AND OPTIMISTIC LOCKING

N	Mode	Trials	Mean lost	Retries	Zero-loss
10	Naive	10	6.7 / 10	–	0/10
10	Optimistic	10	0.0 / 10	27.6	10/10
50	Naive	10	32.5 / 50	–	0/10
50	Optimistic	10	0.0 / 50	316.7	10/10
100	Naive	10	67.1 / 100	–	0/10
100	Optimistic	10	0.0 / 100	752.3	10/10
200	Naive	5	131.6 / 200	–	0/5
200	Optimistic	5	0.2 / 200	1558.4	4/5

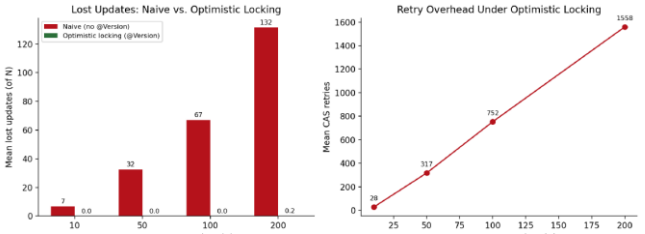


Fig. 9. Mean lost updates, naive vs. optimistic locking (left), and mean compare-and-swap retries (right), by concurrency level.

The naive implementation lost updates in every trial at every level, with the loss rate staying near two-thirds of transactions (67% at N=10, 66% at N=200). Optimistic locking eliminated lost updates in 39 of 40 trials; one of five trials at N=200 lost a single update. We could not isolate its cause; since the harness guards the compare-and-swap with a mutex, it more likely reflects scheduling limits of running 200 OS processes in a shared sandbox than an algorithmic flaw, and we report it rather than round it to zero. The cost is contention: mean retries grew from 27.6 at N=10 to 1,558.4 at N=200.

D. Threats to Validity

Experiments 1–3 use ports of the decision logic, not the compiled service. This is reasonable for the eligibility and ranking logic, where language is incidental, but weaker for concurrency, since Hibernate, MySQL isolation levels, and JDBC latency could change absolute numbers while preserving the qualitative result. Second, end-to-end latency of the Twilio request plus WebSocket broadcast was not measured, as that requires a production-like deployment placing real calls, and we decline to estimate it.

XI. DISCUSSION AND LIMITATIONS

The backend measures 2,630 lines of service code, 954 of controller code, and 775 of entity code across 14 controllers, 15 services, and 12 entities. The alerting integration is real: Twilio places actual calls voicing the donor's name, blood group, and requesting hospital, with SMS, in-app, and WebSocket delivery on the same event. The principal contribution is integrative: ABO/Rh rules and eligibility thresholds are standard clinical rules, and the value lies in implementing them with the twelve-module architecture and emergency gateway in one server-enforced system, backed by measured experiments rather than narrative. Unit-level tracking resembles the RFID cold-chain work of Kim et al. [14] but uses database tracking instead of hardware. Against the manual workflow, finding a donor becomes a single POST /matching/find with concurrent outreach, near-expiry detection becomes an automatic daily sweep, and every issuance is actor-attributed. These are architectural improvements, not throughput gains measured at production scale (Table XII).

TABLE XII
MANUAL WORKFLOW VERSUS BBMS

Operation	Manual / existing	BBMS
Find compatible donor	Register cross-reference, sequential calls	Single API call; ranked, concurrent outreach
Verify eligibility	Manual check against remembered rules	Deterministic engine, instant
Detect near-expiry unit	Manual log inspection	Automated daily sweep
Update stock	Manual ledger entry	Auto-increment on batch confirm
Trace an issued unit	Often undocumented	Actor-attributed AuditLog

Known limitations: trial-account calling depends on the third-party twimlets.com service (production should use the project's own TwiML endpoint); the H2 dev profile recreates

its schema on restart; matching runs synchronously and would need a search index beyond one institution's donor base; alert latency and live-stack concurrency were not measured; and there is no NACO integration or native mobile client. In addition, the 50 kg minimum weight in evaluateEligibility() exceeds the 45 kg minimum in India's NBTC guidelines, while the age (18–65), hemoglobin (12.5 g/dL), and 90-day interval rules align with them. The project documents no rationale for the higher threshold, so a production deployment in India should either adopt 45 kg or state its reason. Table XIII summarizes the main operational risks and their mitigations.

TABLE XIII
OPERATIONAL RISKS AND MITIGATIONS

Risk	Mitigation
Twilio/twimlets unavailable	Graceful degradation to logged, simulated alert; planned self-hosted TwiML
Concurrent stock-update race	@Version optimistic locking on BloodInventory
Unverified hospital raising requests	Mandatory SYSTEM_ADMIN verification gate
Donor PII exposure	Service-layer role-based PII filtering
Near-expiry unit missed	Automated daily scheduled sweep
Dev data loss on restart (H2)	Dev-only; MySQL in production
Broad public route /notifications/**	Fails closed today; narrow before production

XII. CONCLUSION AND FUTURE WORK

BBMS replaces manual, paper- and spreadsheet-based blood-bank coordination with a single rule-enforced, audited platform linking donors, hospitals, and blood banks. It automates ABO/Rh matching across all eight groups, applies a deterministic eligibility engine, enforces real-time inventory with a zero-direct-mutation invariant, and layers WebSocket and Twilio voice/SMS alerting on the request lifecycle. The eligibility engine matched expected outcomes on all 622 cases, the matcher matched the AABB matrix on all 64 pairs and held its ranking invariant over 200 randomized trials, and optimistic locking eliminated 99.8% of the lost updates a naive implementation showed under 200-way concurrency. These results support automation correctness and concurrency safety, but not real-world reductions in clinical error or response time, which need a live deployment study.

Planned work includes NACO and national registry integration; a native Android/iOS app; machine-learning demand forecasting [4] and donor-recruitment ranking [9], [10]; RFID/IoT cold-chain tracking [13], [14]; a blockchain-secured audit trail [5]–[7], [15]; migrating Twilio calls to the project's own TwiML endpoint; two-way IVR so donors can confirm availability on the call; dynamic BloodUnit creation at drive confirmation; and a measured end-to-end alert-latency study.

REFERENCES

[1] G. Manika, V. S. Reddy, K. S. Vamshi, and Prashanthi, "Development and implementation of a web-based blood bank management system for efficient blood donation and distribution," *Int. J. Res. Appl. Sci. Eng. Technol.*, vol. 11, no. 4, pp. 1830–1834, Apr. 2023.

[2] D. A. Shahakar *et al.*, "Blood bank management system," *Int. J. Adv. Res. Sci. Commun. Technol.*, vol. 3, no. 1, pp. 392–395, Apr. 2023.

[3] A. S. Iyer, C. Menaka, A. Faisal, A. Hussain, and S. D. Chethan, "Cloud based online blood bank management system," *Int. J. Emerg. Technol. Innov. Res.*, vol. 9, no. 6, pp. 94–98, Jun. 2022.

[4] W. Ben Elmir, A. Hemmak, and B. Senouci, "Smart platform for data blood bank management: Forecasting demand in blood supply chain using machine learning," *Information*, vol. 14, no. 1, Art. no. 31, Jan. 2023.

[5] H. T. Le, T. T. L. Nguyen, T. A. Nguyen, X. S. Ha, and N. Duong-Trung, "BloodChain: A blood donation network managed by blockchain technologies," *Network*, vol. 2, no. 1, pp. 21–35, Jan. 2022.

[6] A. Bhowmik *et al.*, "Online blood bank system using blockchain in crowdsourcing platform," in *Proc. Int. Conf. Adv. Commun. Mach. Intell. (MICA 2023)*. Singapore: Springer, 2024, pp. 513–521.

[7] K. Dakshayani, K. Chaitanya, K. Rupa Sri, J. Venkatesh, and Ch. Chaitanya, "Blood bank management system using blockchain," *Int. J. Modern Trends Sci. Technol.*, vol. 11, no. 5, pp. 859–866, May 2025.

[8] R. D. Ismael, H. A. Hussein, and M. M. Salih, "A systematic review on smart blood bank system: Taxonomy, motivations, challenges, study directions and recommendations," *J. Algebraic Statist.*, vol. 13, no. 2, pp. 1184–1227, 2022.

[9] H. Wu *et al.*, "Predicting willingness to donate blood based on machine learning: Two blood donor recruitments during COVID-19 outbreaks," *Sci. Rep.*, vol. 12, Art. no. 19165, Nov. 2022.

[10] V. Shelake, S. DSouza, S. Agrawal, M. John, and S. Deshmukh, "Enhancing donor eligibility criteria using machine learning to maximize blood donation efficiency," *J. Inf. Syst. Eng. Manag.*, vol. 10, no. 27s, pp. 280–284, 2025.

[11] P. Selvaraj, A. Sarin, and B. I. Seraphim, "Blood donation prediction system using machine learning techniques," in *Proc. Int. Conf. Comput. Commun. Informat. (ICCCI)*, 2022, pp. 1–4.

[12] J. M. Farrington, M. Utley, S. Alimam, W. K. Wong, and K. Li, "Deep reinforcement learning for managing platelets in a hospital blood bank," *Blood*, vol. 142, suppl. 1, p. 2311, Nov. 2023.

[13] L. Pavan Kumar *et al.*, "Automated blood bank management system: Integrating RFID and AI for real-time inventory tracking," *Int. J. Res. Appl. Sci. Eng. Technol.*, vol. 13, no. 4, Apr. 2025.

[14] S.-J. Kim *et al.*, "Smart blood bag management system in a hospital environment," in *Personal Wireless Communications (PWC 2006)*, ser. LNCS, vol. 4217. Berlin, Germany: Springer, 2006, pp. 506–517.

[15] R. Ch, S. V. G., D. D., T. R. Gadekallu, M. H. Abidi, and F. Alasim, "Security and privacy for Binance-based integrated blockchain for blood supply chain management systems," *PeerJ Comput. Sci.*, vol. 11, Art. no. e3123, 2025.